

第七讲：Krusell–Smith 方法

异质性主体宏观经济学的计算方法

孙振越 (Jeffrey Sun)

2026 年 5 月 19 日

多伦多大学

Krusell-Smith 方法

设定：带总量 TFP 冲击的 Aiyagari 模型

总量 TFP $Z_t \in \{Z_{\text{bad}}, Z_{\text{good}}\}$ 服从转移矩阵为 Π_Z 的马尔可夫链。

特异性状态：(z_{it}, b_{it})

- 生产率 $z_{it} \in \{z_{\text{lo}}, z_{\text{hi}}\}$ ，服从转移矩阵为 Π_z 的马尔可夫链
- 财富 b_{it}

技术：带 TFP Z_t 的 Cobb–Douglas 生产函数：

$$Y_t = Z_t K_t^\alpha L^{1-\alpha}.$$

假设 L 恒定，等于 z 的平稳均值。

价格

毛利率与工资由 Z_t, K_t, L 给定,

$$R(Z_t, K_t) = 1 + \alpha Z_t (K_t/L)^{\alpha-1} - \delta,$$

$$w(Z_t, K_t) = (1 - \alpha) Z_t (K_t/L)^\alpha.$$

问题：家庭的价格预测既**影响**未来的 K_{t+1} ，又**依赖于**它。

K_{t+1} 依赖于 Λ_t

$$K_{t+1} = \int b'_i(b_i, z_i; Z_t, K_t) d\Lambda_t(b_i, z_i).$$

策略函数 $b'_i(b_i, z_i; Z_t, K_t)$ 依赖于价格预测。

价格 R_{t+s}, w_{t+s} 的预测通过 K_{t+s} 依赖于 Λ_{t+s} 。

注意：来自不同 Λ_t 的相同 K_t 可能导出不同的 K_{t+1} 。

贝尔曼方程

分布 Λ 成为状态变量：

$$V(b_i, z_i; \Lambda, Z) = \max_{c_i, b'_i} u(c_i) + \beta \mathbb{E}[V(b'_i, z'_i; \Lambda', Z') \mid z_i, Z],$$

Λ 是无限维的

Λ_t 是定义在所有 (b, z) 上的**分布**。

“真实”的 Λ_t 是无限维的。

即便我们将 Λ_t 离散化为一个矩阵，**每个网格点都是一个状态变量**。

当财富网格点数 $N_b = 80$ 、生产率水平数 $\#z = 2$ 时，网格共有 160 个点。

在所有可能的 Λ 值上构建 V 网格： 5^{160-1} 个单元，约为 10^{111} 。

对 $V(\dots; \dots, \Lambda)$ 进行可处理的参数化

两种思路：

- (a) 降低 Λ 的维度。(Krusell-Smith。)用 Λ_t 的矩来代替整个分布。
- (b) 降低表示的复杂度。(nVFI。)用一族灵活的函数（例如神经网络）对 $V(\Lambda, \cdot)$ 进行参数化。

Krusell–Smith 方法

微观基础：有限理性

Krusell and Smith (1998)：假设**家庭**只观测到 Λ_t 的矩，而非整个分布。

即，求解 $V(b, z; Z, K)$ 的模型，而非 $V(b, z; Z, \Lambda)$ 的模型。

近似运动方程 (ALM)

假设家庭对 K 使用如下预测规则：

$$\log K_{t+1} = a_0(Z_t) + a_1(Z_t) \log K_t.$$

每个 Z 状态有两个系数。 对于 2 状态的 Z ，共四个数。

对数线性。 K-S 发现这实际上拟合得非常好。

有限理性的贝尔曼方程

在状态中用 K 替换 Λ :

$$V(b_i, z_i, Z, K) = \max_{c_i, b'_i} u(c_i) + \beta \mathbb{E}[V(b'_i, z'_i, Z', K')],$$

约束条件为

$$c_i + b'_i = R(Z, K) b_i + w(Z, K) z_i,$$

主体的信念为

$$K' = \exp(a_0(Z) + a_1(Z) \log K),$$

且 Z' 从 Π_Z 中抽取。

行为链（五个阶段）

在近似运动方程（ALM）的假设下，我们只需向家庭模块添加两个阶段即可求解模型：

1. z_shock : $z_{it} \rightarrow z_i^{t+1}$ （在 z 上的马尔可夫转移）。
2. $income$: $b'_i \leftarrow R b_i + w z_i$ ，其中 (R, w) 由 (Z, K) 给出。
3. $savings$: 选择 b_i^{t+1} 。
4. K_evolve : $K_t \rightarrow K_{t+1}$ 。
5. Z_shock : $Z_t \rightarrow Z_{t+1}$ 。

$$z_shock \circ income \circ savings \circ K_evolve \circ Z_shock.$$

求解这条链便给出了在给定 $a_0(\cdot), a_1(\cdot)$ 下家庭的**行为**，并使我们能够向前模拟**真实模型**。

K-S 外层循环

1. **选取**一个初始 ALM $(a_0^{(0)}, a_1^{(0)})$ 。
2. 在当前 ALM 下**求解**四维家庭 VFI。
3. 在实现的 $\{Z_t\}$ 与真实加总 $K_t = \int b d\Lambda_t$ 下**模拟**一个长时段经济。
4. 通过将 $\log K_{t+1}$ 对 $\log K_t$ 做 OLS（按 Z 分别进行）来**重新拟合** (a_0, a_1) 。
5. **迭代**直至 ALM 系数不再变动。

实现 Krusell–Smith

参数

Krusell-Smith 校准 (1998):

- $\beta = 0.96$, $\gamma = 1$ (对数效用), $\alpha = 0.36$, $\delta = 0.025$ 。
- $Z \in \{0.99, 1.01\}$, Π_Z 具有持续性 (对角线上为 0.875)。
- $z \in \{0.07, 1.0\}$ 。

网格 (为示例起见取较小规模):

- $N_b = 80$, 对数等距分布于 $[0, 80]$ 。
- $N_K = 5$, 线性分布于 $[10.5, 14.5]$ 。
- 单元数: $80 \times 2 \times 2 \times 5 = 1600$ 。

状态布局

```
layout = StateLayout(  
    StateAxis(:wealth, continuous_grid(p.b_min, p.b_max; length=p.N_b, spacing=:log)),  
    StateAxis(:z,      p.z_grid),  
    StateAxis(:Z_idx,  [1, 2]),  
    StateAxis(:K,      continuous_grid(p.K_min, p.K_max; length=p.N_K)),  
)
```

- 两个连续轴 (:wealth、:K)，两个类别轴 (:z、:Z_idx)。

阶段 1–2: z_shock 与 income

阶段 1–2

```
z_shock = MarkovStage(layout; axis=:z, transition=p.Π_z)

income = WealthChangeStage(layout; wealth_post=(cell; env) -> begin
    Z = p.Z_vals[cell.Z_idx]
    (;R, w) = ks_prices(cell.K, Z, p)
    R * cell.wealth + w * cell.z
end)
```

注意。这从单元坐标中读取 (Z, K) 。

阶段 3: ConsumptionSavings

阶段 3

```
savings = ConsumptionSavingsStage(layout;  $\beta = p.\beta$ ,  
  utility=(cell, c; env) -> u_crra(c, Val(p. $\gamma$ )),  
  monotone_search=:divide_conquer)
```

- 分治搜索利用了关于财富的单调性。

阶段 4–5: K_evolve 与 Z_shock

阶段 4–5

```
K_evolve = WealthChangeStage(layout; wealth_axis=:K,  
    wealth_post=(cell; env) ->  
        exp(env.a0[cell.Z_idx] + env.a1[cell.Z_idx] * log(cell.K)))  
  
Z_shock = MarkovStage(layout; axis=:Z_idx, transition=p.Π_Z)
```

构建家庭模块

```
chain = z_shock ◦ income ◦ savings ◦ K_evolve ◦ Z_shock

hh = define_moments!(chain;
    K_supplied=at_end(integrand=:wealth, reduce=sum))
```

内层求解：给定 $a_0(\cdot), a_1(\cdot)$ 的 VFI

```
a0_init = [log(K_bar), log(K_bar)]  
a1_init = [0.0, 0.0]  
env0 = make_env(hh_4d; a0=a0_init, a1=a1_init)  
  
res4d = solve_steady_state_given_env!(hh_4d, env0; lambda_tol=1e-5, lambda_maxiter=50_000)
```

初始猜测。一个常数 ALM：对两个 Z 状态均有 $K_{t+1} = \bar{K}$ ($a_1 \equiv 0$, $a_0 \equiv \log \bar{K}$)。

模拟用家庭模块

在模拟时，我们想要的是实现的 (Z_t, K_t) ，而非主体的信念。

行为链中的 `K_evolve` 与 `Z_shock` 编码的是信念，而非 K 的真实运动方程。

真实的链只包含前三个阶段 (`z_shock` `income` `savings`)，而 Z, K 在**外层循环**中更新。

模拟真实模型

每一期：

1. 计算 $K_t = \sum_{b,z} b \Lambda_t[b, z]$ 。
2. 将 Λ_t 提升为四维分布，把 (Z_t, K_t) 添加为“特异性”状态变量。
3. 使用行为链（5 个阶段）模拟该四维分布。
4. 对 (Z, K) 求边际，得到 (b, z) 上的 Λ_{t+1} 。
5. 从 Π_Z 中抽取 $Z_{t+1} \mid Z_t$ 。

用 OLS 重新拟合 ALM

```
function regress_ALM(K_path, Z_idx_path, z_idx, window)
    rows = [t for t in window if Z_idx_path[t] == z_idx]
    x = log.(K_path[rows])
    y = log.(K_path[rows .+ 1])
    # OLS:  $y = a_0 + a_1 * x$  ;  $R^2 = 1 - SS_{res}/SS_{tot}$ 
    ...
    return (; a0, a1, R^2, n=length(rows))
end

fit_bad = regress_ALM(sim0.K_path, sim0.Z_idx_path, 1, window_fit)
fit_good = regress_ALM(sim0.K_path, sim0.Z_idx_path, 2, window_fit)
```

外层循环

(内容过长, 无法放入幻灯片; 详见 notebook。)

```
function ks_iterate(p::KSPParams, K_bar,  $\Lambda_0$ ; update_speed=0.5, rtol=1e-3, maxiter=20, T_sim
    =2000, T_burn=500)
    hh_4d = behavioural_alm_household(p)
    a0 = [log(K_bar), log(K_bar)]
    a1 = [0.0, 0.0]
    err, iters = Inf, 0
    while err >= rtol
        env = make_env(hh_4d; a0, a1)
        solve_steady_state_given_env!(hh_4d, env; lambda_tol=1e-5, lambda_maxiter=50_000)

        sim = ks_simulate(hh_4d, p; T=T_sim,  $\Lambda_{\text{init}}=\Lambda_0$ , Z_init=1)
        window_fit = (T_burn + 1):(T_sim - 1)
        fit_bad = regress_ALM(sim.K_path, sim.Z_idx_path, 1, window_fit)
        fit_good = regress_ALM(sim.K_path, sim.Z_idx_path, 2, window_fit)

        a0_new = [fit_bad.a0, fit_good.a0]
        a1_new = [fit_bad.a1, fit_good.a1]
```